

**LAPORAN PRAKTIKUM STRUKTUR DATA**  
**ALGORITMA SORTING PADA JAVA**

Disusun Oleh :

Rahmat Ananda Nazar

2511532008

Dosen Pengampu : Dr. Wahyudi, S.T M.T

Asisten Praktikum : Muhammad Galid Avero



**DEPARTEMEN INFORMATIKA**  
**FAKULTAS TEKNOLOGI INFORMASI**  
**UNIVERSITAS ANDALAS**

**2026**

## **KATA PENGANTAR**

Puji syukur atas kehadiran Tuhan Yang Maha Esa berkat rahmat dan karunianya penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 8 dengan judul “Algoritma Sorting pada Java” ini dengan baik. Pada laporan praktikum pekan satu ini akan membahas mengenai bagaimana cara kerja beberapa algoritma sorting (merge sort, quick sort dan shell sort) dan menerapkannya untuk membuat sebuah program sorting sederhana.

Penulis menyadari bahwa laporan ini tidak terlepas dari kesalahan dan kekurangan baik dari penulisan dan pembahasan. Oleh karena itu penulis ingin mengucapkan terimakasih kepada dosen pengampu Bapak Dr. Wahyudi, S.T M.T. dan asisten praktikum Muhammad Galid Avero yang telah memberikan arahan selama proses praktikum dan proses penyusunan laporan praktikum. Penulis berharap laporan ini dapat bermanfaat bagi pembaca dalam memahami algoritma sorting pada java

## DAFTAR ISI

|                                |    |
|--------------------------------|----|
| KATA PENGANTAR.....            | i  |
| DAFTAR ISI .....               | ii |
| BAB I PENDAHULUAN .....        | 1  |
| 1.1 Latar Belakang .....       | 1  |
| 1.2 Tujuan Praktikum .....     | 1  |
| BAB II PEMBAHASAN .....        | 2  |
| 2.1 Shell Sort.....            | 2  |
| 2.1.1 Penjelasan Program ..... | 2  |
| 2.1.2 Kode Program .....       | 3  |
| 2.2 Merge Sort.....            | 3  |
| 2.2.1 Penjelasan Program ..... | 3  |
| 2.2.2 Kode Program .....       | 4  |
| 2.3 Quick Sort .....           | 5  |
| 2.3.1 Penjelasan Program ..... | 5  |
| 2.3.2 Kode Program .....       | 6  |
| BAB III PENUTUP .....          | 7  |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Pengurutan data (sorting) merupakan salah satu operasi paling mendasar dan penting dalam bidang ilmu komputer, khususnya dalam manajemen basis data dan optimasi pencarian. Pada praktikum sebelumnya, telah dipelajari algoritma sorting dasar seperti Bubble Sort, Insertion Sort, dan Selection Sort yang memiliki kompleksitas waktu rata-rata. Meskipun mudah diimplementasikan, algoritma tersebut kurang efisien untuk menangani data dalam skala besar.

Untuk mengatasi keterbatasan tersebut, dikembangkan algoritma pengurutan lanjutan yang menggunakan pendekatan lebih optimis dan efisien. Shell Sort hadir sebagai pengembangan dari Insertion Sort dengan membandingkan elemen dengan jarak (gap) tertentu. Sementara itu, Quick Sort dan Merge Sort menerapkan paradigma Divide and Conquer (bagi dan taklukkan) untuk memecah masalah besar menjadi sub-masalah yang lebih kecil, sehingga mampu mereduksi kompleksitas waktu secara signifikan menjadi pada kondisi optimal. Pemahaman mendalam mengenai struktur kode dan visualisasi pergerakan data di memori untuk ketiga algoritma ini sangat krusial bagi mahasiswa Informatika

### **1.2 Tujuan Praktikum**

- Mahasiswa memahami prinsip dasar dan mekanisme kerja algoritma Shell Sort, Quick Sort, dan Merge Sort.
- Mahasiswa mampu mengimplementasikan algoritma pengurutan lanjutan tersebut ke dalam bahasa pemrograman Java.
- Mahasiswa mampu menganalisis perbedaan efisiensi penukaran data dan alokasi memori dari masing-masing algoritma.

## **BAB II**

### **PEMBAHASAN**

#### **2.1 Shell Sort**

##### **2.1.1 Penjelasan Program**

Algoritma Shell Sort bekerja dengan membandingkan elemen-elemen yang terpisah oleh jarak (gap) tertentu, yang nilainya terus berkurang setengah di setiap iterasi luar hingga mencapai nilai Inisialisasi Gap. Pada awal eksekusi, nilai gap ditentukan berdasarkan panjang array dibagi dua ( $n / 2$ ). Proses Pergeseran: Di dalam perulangan for, elemen pada indeks  $i$  (dimulai dari indeks gap) disimpan ke dalam variabel temporer key. Pengecekan Kondisi Variabel  $j$  akan memeriksa elemen-elemen sebelumnya yang berjarak sejauh gap. Jika elemen  $arr[j - gap]$  lebih besar daripada key, maka elemen tersebut digeser ke kanan menempati posisi  $j$ . Penempatan Elemen Proses pergeseran berhenti ketika posisi yang tepat ditemukan atau indeks  $j$  sudah lebih kecil dari nilai gap. Nilai key kemudian dimasukkan ke dalam  $arr[j]$ . Proses ini mengulangi logika Insertion Sort namun dengan lompatan indeks yang membuat elemen-elemen besar lebih cepat berpindah ke posisi kanan memori.

### 2.1.2 Kode Program

```
public class ShellSort_2511532008 {
    public static void shellSort_2008(int[] A_2008) {
        int n_2008 = A_2008.length;
        int gap_2008 = n_2008 / 2;
        while (gap_2008 > 0) {
            for (int i_2008 = gap_2008; i_2008 < n_2008; i_2008++) {
                int temp_2008 = A_2008[i_2008];
                int j_2008 = i_2008;
                while (j_2008 >= gap_2008 && A_2008[j_2008 - gap_2008] > temp_2008) {
                    A_2008[j_2008] = A_2008[j_2008 - gap_2008];
                    j_2008 = j_2008 - gap_2008;
                }
                A_2008[j_2008] = temp_2008;
            }
            gap_2008 = gap_2008 / 2;
        }
    }

    public static void main(String[] args) {
        int[] data_2008 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
        System.out.print("Sebelum: ");
        printArray_2008(data_2008);
        shellSort_2008(data_2008);
        System.out.print("Sesudah (Shell Sort) : ");
        printArray_2008(data_2008);
    }

    public static void printArray_2008(int[] arr_2008) {
        for (int i_2008 : arr_2008) System.out.print(i_2008 + " ");
        System.out.println();
    }
}
```

## 2.2 Merge Sort

### 2.2.1 Penjelasan Program

Merge Sort menjamin stabilitas pengurutan dengan membagi array menjadi dua bagian secara konsisten hingga berukuran satu elemen, lalu menggabungkannya kembali secara terurut. Pembagian (Divide) Program mencari titik tengah  $m = l + (r - l) / 2$ . Secara rekursif, sub-array dipecah menjadi bagian kiri `mergeSort(arr, l, m)` dan bagian kanan `mergeSort(arr, m + 1, r)`. Alokasi Memori Tambahan Pada metode merge, program membuat dua array temporer baru, yaitu `L[]` (Kiri) dan `R[]` (Kanan). Data dari array utama disalin ke dalam kedua array pembantu ini. Penggabungan Terurut (Conquer/Merge) Tiga buah pointer (`i` untuk `L`, `j` untuk `R`, dan `k` untuk array asli `arr`) digunakan untuk membandingkan elemen terkecil antara `L[i]` dan `R[j]`. Elemen yang lebih kecil akan dimasukkan kembali ke array utama `arr[k]`. Pembersihan Sisa Elemen Dua perulangan `while` terakhir berfungsi untuk menyalin sisa elemen dari `L` atau `R` yang belum sempat dimasukkan akibat salah satu array pembantu sudah habis terlebih dahulu.

## 2.2.2 Kode Program

```
public class MergeSort_2511532008 {
    void merge_2008(int[] arr_2008, int l_2008, int m_2008, int r_2008) {
        int n1_2008 = m_2008 - l_2008 + 1;
        int n2_2008 = r_2008 - m_2008;
        int[] L_2008 = new int[n1_2008];
        int[] R_2008 = new int[n2_2008];
        for (int i_2008 = 0; i_2008 < n1_2008; ++i_2008) {
            L_2008[i_2008] = arr_2008[l_2008 + i_2008];
        }
        for (int j_2008 = 0; j_2008 < n2_2008; ++j_2008) {
            R_2008[j_2008] = arr_2008[m_2008 + 1 + j_2008];
        }
        int i_2008 = 0, j_2008 = 0;
        int k_2008 = l_2008;
        while (i_2008 < n1_2008 && j_2008 < n2_2008) {
            if (L_2008[i_2008] <= R_2008[j_2008]) {
                arr_2008[k_2008] = L_2008[i_2008];
                i_2008++;
            } else {
                arr_2008[k_2008] = R_2008[j_2008];
                j_2008++;
            }
            k_2008++;
        }
        while (i_2008 < n1_2008) {
            arr_2008[k_2008] = L_2008[i_2008];
            i_2008++;
            k_2008++;
        }
        while (j_2008 < n2_2008) {
            arr_2008[k_2008] = R_2008[j_2008];
            j_2008++;
            k_2008++;
        }
    }
    void sort_2008(int[] arr_2008, int l_2008, int r_2008) {
        if (l_2008 < r_2008) {
            int m_2008 = (l_2008 + r_2008) / 2;
            sort_2008(arr_2008, l_2008, m_2008);
            sort_2008(arr_2008, m_2008 + 1, r_2008);
            merge_2008(arr_2008, l_2008, m_2008, r_2008);
        }
    }
    static void printArray_2008(int[] arr_2008) {
        int n_2008 = arr_2008.length;
        for (int i_2008 = 0; i_2008 < n_2008; ++i_2008) {
            System.out.print(arr_2008[i_2008] + " ");
        }
        System.out.println();
    }
    public static void main(String[] args) {
        int[] arr_2008 = { 12, 11, 13, 5, 6, 7 };
        System.out.println("Sebelum terurut :");
        printArray_2008(arr_2008);
        MergeSort_2511532008 ob_2008 = new MergeSort_2511532008();
        ob_2008.sort_2008(arr_2008, 0, arr_2008.length - 1);
        System.out.println("\nSesudah Terurut menggunakan Merge Sort : ");
        printArray_2008(arr_2008);
    }
}
```

## **2.3 Quick Sort**

### **2.3.1 Penjelasan Program**

Quick Sort menggunakan teknik partisi dan rekursif dengan memilih satu elemen sebagai pivot (pada kode di atas menggunakan elemen terakhir `arr[high]`). Fungsi Partisi Fungsi `partition` bertujuan untuk memposisikan semua elemen yang lebih kecil dari pivot ke sebelah kiri, dan elemen yang lebih besar ke sebelah kanan. Pointer `i` digunakan untuk menandai batas elemen yang lebih kecil dari pivot. Proses Swap di Memori Melalui perulangan `j`, setiap kali ditemukan elemen `arr[j]` yang lebih kecil dari pivot, nilai pointer `i` akan bertambah, kemudian terjadi pertukaran (swap) data antara `arr[i]` dan `arr[j]`. Penempatan Pivot Akhir Setelah perulangan selesai, pivot ditempatkan di posisi tengah yang benar, yaitu pada indeks `i + 1` melalui operasi swap dengan `arr[high]`. Pemanggilan Rekursif Setelah fungsi partisi mengembalikan indeks posisi pivot (`pi`), metode `quickSort` memanggil dirinya sendiri secara rekursif untuk mengurutkan sub-array kiri (`low`, `pi - 1`) dan sub-array kanan (`pi + 1`, `high`) hingga seluruh array terurut sempurna di dalam memori stack.



### 2.3.2 Kode Program

```
public class QuickSort_2511532008 {
    static void swap_2008(int[] arr_2008, int i_2008, int j_2008) {
        int temp_2008 = arr_2008[i_2008];
        arr_2008[i_2008] = arr_2008[j_2008];
        arr_2008[j_2008] = temp_2008;
    }
    static void medianOfThree_2008(int[] arr_2008, int low_2008, int high_2008) {
        int mid_2008 = low_2008 + (high_2008 - low_2008) / 2;
        if (arr_2008[low_2008] > arr_2008[mid_2008]) {
            swap_2008(arr_2008, low_2008, mid_2008);
        }
        if (arr_2008[low_2008] > arr_2008[high_2008]) {
            swap_2008(arr_2008, low_2008, high_2008);
        }
        if (arr_2008[mid_2008] > arr_2008[high_2008]) {
            swap_2008(arr_2008, mid_2008, high_2008);
        }
        swap_2008(arr_2008, mid_2008, high_2008);
    }
    static int partition_2008(int[] arr_2008, int low_2008, int high_2008) {
        medianOfThree_2008(arr_2008, low_2008, high_2008);
        int pivot_2008 = arr_2008[high_2008];
        int i_2008 = (low_2008 - 1);
        for (int j_2008 = low_2008; j_2008 <= high_2008 - 1; j_2008++) {
            if (arr_2008[j_2008] < pivot_2008) {
                i_2008++;
                swap_2008(arr_2008, i_2008, j_2008);
            }
        }
        swap_2008(arr_2008, i_2008 + 1, high_2008);
        return (i_2008 + 1);
    }
    static void quickSort_2008(int[] arr_2008, int low_2008, int high_2008) {
        if (low_2008 < high_2008) {
            int pi_2008 = partition_2008(arr_2008, low_2008, high_2008);
            quickSort_2008(arr_2008, low_2008, pi_2008 - 1);
            quickSort_2008(arr_2008, pi_2008 + 1, high_2008);
        }
    }
    public static void printArr_2008(int[] arr_2008) {
        for (int i_2008 = 0; i_2008 < arr_2008.length; i_2008++) {
            System.out.print(arr_2008[i_2008] + " ");
        }
        System.out.println();
    }
    public static void main(String[] args) {
        int[] arr_2008 = { 10, 7, 8, 9, 1, 5 };
        int N_2008 = arr_2008.length;
        System.out.print("Data sebelum diurutkan : ");
        printArr_2008(arr_2008);
        quickSort_2008(arr_2008, 0, N_2008 - 1);
        System.out.print("Data Terurut quicksort : ");
        printArr_2008(arr_2008);
    }
}
```

### **BAB III**

### **PENUTUP**

Berdasarkan hasil implementasi dan pengujian terhadap ketiga kode program algoritma pengurutan lanjutan yang telah dibuat, dapat disimpulkan bahwa: Shell Sort merupakan optimasi dari Insertion Sort yang sangat efisien untuk data berukuran menengah karena berhasil memangkas langkah pergeseran data yang jauh menggunakan sistem jarak (gap). Quick Sort memiliki performa kecepatan pertukaran data yang sangat tinggi di dalam memori lokal karena melakukan pengurutan secara langsung pada array asli (in-place sorting) memanfaatkan elemen pivot. Namun, kinerjanya bergantung pada pemilihan nilai pivot tersebut. Merge Sort memiliki karakteristik alokasi memori yang lebih intensif karena membutuhkan pembuatan array temporer baru (L dan R) di setiap proses penggabungan. Meskipun demikian, algoritma ini memberikan jaminan performa waktu yang stabil dalam kondisi data acak maupun terburuk sekalipun (worst-case).

## TUGAS PRAKTIKUM

```
import java.util.Scanner;

class Lagu {
    String judul;
    String penyanyi;
    int durasi;

    public Lagu(String judul, String penyanyi, int durasi) {
        this.judul = judul;
        this.penyanyi = penyanyi;
        this.durasi = durasi;
    }
}

public class Sorting_2511532008 {
    private Lagu[] dataLagu_2008 = new Lagu[20];
    private int jumlahLagu_2008 = 0;
    public void inputData_2008() {
        Scanner scanner = new Scanner(System.in);
        System.out.println("=== Input Data Lagu (Minimal 7, Maksimal 20) ===");
        while (jumlahLagu_2008 < 20) {
            System.out.print("\nMasukkan judul lagu (atau ketik 'selesai' jika sudah min. 7 lagu): ");
            String judul = scanner.nextLine();
            if (judul.equalsIgnoreCase("selesai")) {
                if (jumlahLagu_2008 < 7) {
                    System.out.println("Gagal selesai. Anda harus memasukkan minimal " + (7 - jumlahLagu_2008) + " lagu lagi.");
                    continue;
                }
                break;
            }
            System.out.print("Masukkan nama penyanyi: ");
            String penyanyi = scanner.nextLine();
            System.out.print("Masukkan durasi (detik): ");
            int durasi = scanner.nextInt();
            scanner.nextLine();
            dataLagu_2008[jumlahLagu_2008] = new Lagu(judul, penyanyi, durasi);
            jumlahLagu_2008++;
        }
    }

    public void mergeSort_2008(Lagu[] arr, int kiri, int kanan) {
        if (kiri < kanan) {
            int tengah = kiri + (kanan - kiri) / 2;
            mergeSort_2008(arr, kiri, tengah);
            mergeSort_2008(arr, tengah + 1, kanan);
            merge_2008(arr, kiri, tengah, kanan);
        }
    }
}
```

```

    }
    private void merge_2008(Lagu[] arr, int kiri, int tengah, int kanan) {
        int n1 = tengah - kiri + 1;
        int n2 = kanan - tengah;
        Lagu[] L = new Lagu[n1];
        Lagu[] R = new Lagu[n2];
        for (int i = 0; i < n1; ++i) L[i] = arr[kiri + i];
        for (int j = 0; j < n2; ++j) R[j] = arr[tengah + 1 + j];
        int i = 0, j = 0;
        int k = kiri;
        while (i < n1 && j < n2) {
            if (L[i].judul.compareToIgnoreCase(R[j].judul) <= 0) {
                arr[k] = L[i];
                i++;
            } else {
                arr[k] = R[j];
                j++;
            }
            k++;
        }
        while (i < n1) {
            arr[k] = L[i];
            i++;
            k++;
        }
        while (j < n2) {
            arr[k] = R[j];
            j++;
            k++;
        }
    }
}

public void tampilData_2008(String status) {
    System.out.println("\nData " + status + ":");
    for (int i = 0; i < jumlahLagu_2008; i++) {
        System.out.println((i + 1) + ". " + dataLagu_2008[i].judul + " - " + dataLagu_2008[i].penyanyi + " (" + dataLagu_2008[i].durasi + " detik)");
    }
}

public static void main(String[] args) {
    Sorting_2511532008 program = new Sorting_2511532008();
    Scanner input = new Scanner(System.in);
    program.inputData_2008();
    System.out.println("\n=== Sorting Playlist MIM: 2511532008 ===");
    System.out.print("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
    int pilihan = input.nextInt();

    if (pilihan == 3) {
        program.tampilData_2008("Sebelum Sorting");
        program.mergesort_2008(program.dataLagu_2008, 0, program.jumlahLagu_2008 - 1);
        program.tampilData_2008("Setelah Merge Sort (Judul A-Z)");
    } else {
        System.out.println("Algoritma tersebut tidak diimplementasikan. Program ini khusus Merge Sort (Pilihan 3).");
    }

    input.close();
}
}

```